密　级：　　公开

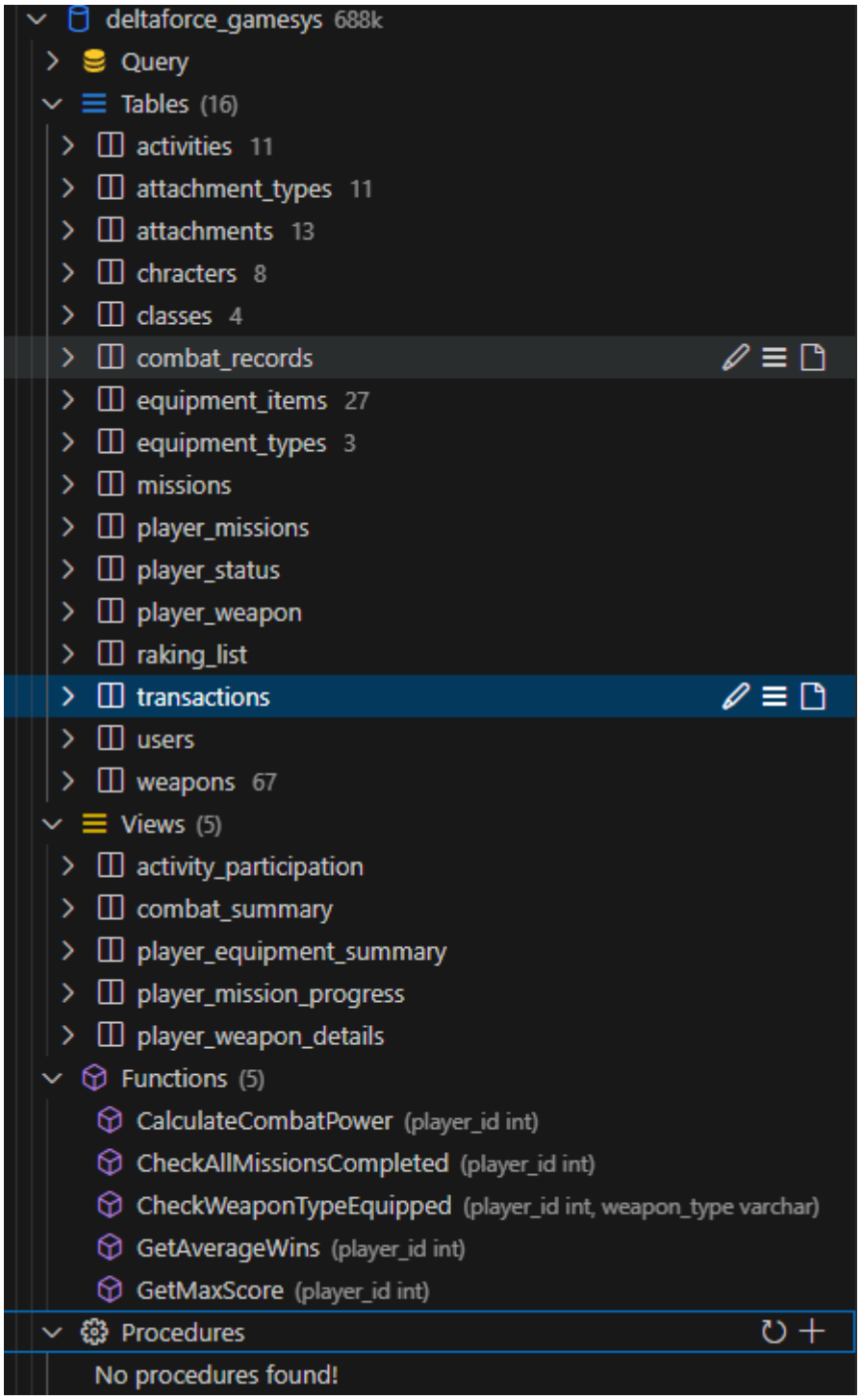
文件编号

Mysql 期末项目

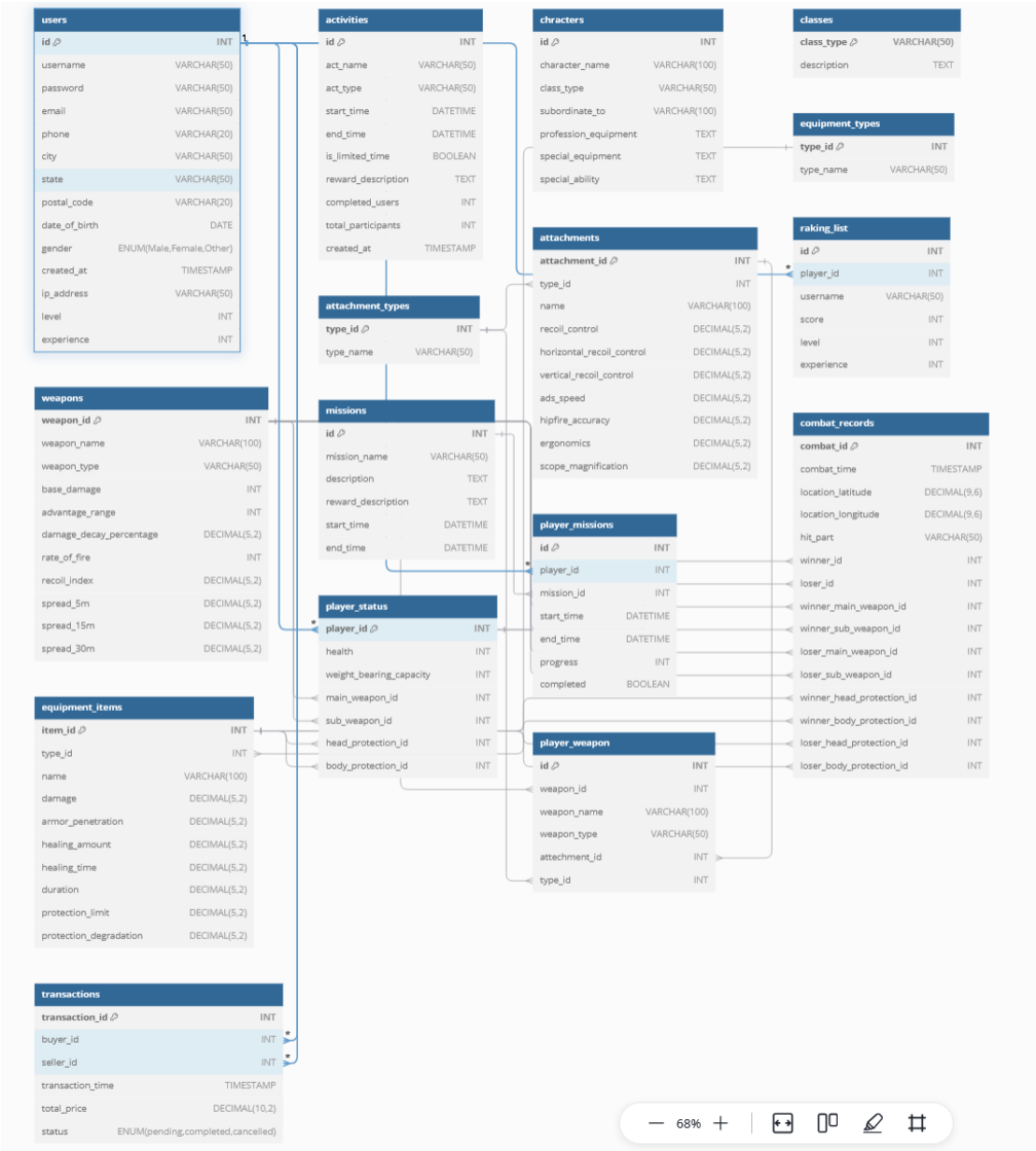
学　　校：	中央民族大学
学　　院：	海南国际学院
专　　业：	数据科学与大数据技术
年　　级：	2023 级 大数据 1 班
姓　　名：	胡可秦
学　　号：	23160133
日　　期：	2025/1/11

一、 数据库设计

总览：这是一个模拟最近大火的国产 fps 网游《三角洲行动》的数据库系统设计



E-R 图：



由 https://dbdiagram.io/d 生成

1. 玩家用户信息表（users）

【设计理由】存储用户的基本信息，包括用户名、密码、电子邮件、电话、地址、出生日期、性别、IP 地址、等级和经验值。

字段名	数据类型	说明
id	INT	用户唯一标识（主键）
username	VARCHAR(50)	用户名

字段名	数据类型	说明
password	VARCHAR(50)	密码
email	VARCHAR(50)	电子邮件
phone	VARCHAR(20)	电话
city	VARCHAR(50)	城市
state	VARCHAR(50)	州/省
postal_code	VARCHAR(20)	邮政编码
date_of_birth	DATE	出生日期
gender	ENUM	性别（Male, Female, Other）
created_at	TIMESTAMP	创建时间
ip_address	VARCHAR(50)	IP 地址
level	INT	等级
experience	INT	经验值

测试数据：

```
SELECT * FROM users LIMIT 100
```

搜索结果集 导出 耗时: 6ms 1 Total 11

id int	username varchar(50)	password varchar(50)	email varchar(50)	phone varchar(20)	city varchar(50)	state varchar(50)	postal_code varchar(20)	date_of_birth date	gender enum('Male','Female',	created_at timestamp	ip_address varchar(50)	level int	experience int
1	Alice	password123	alice@example.com	1234567890	New York	NY	10001	1990-01-01	Female	2025-01-11 21:50:33	192.168.1.1	1	500
2	Bob	password456	bob@example.com	2345678901	Los Angeles	CA	90001	1985-02-15	Male	2025-01-11 21:50:33	192.168.1.2	2	1500
3	Charlie	password789	charlie@example.com	3456789012	Chicago	IL	60601	1992-03-20	Male	2025-01-11 21:50:33	192.168.1.3	3	2500
4	Daisy	pass1234	daisy@example.com	4567890123	Houston	TX	77001	1995-04-25	Female	2025-01-11 21:50:33	192.168.1.4	4	4000
5	Ethan	pass5678	ethan@example.com	5678901234	Phoenix	AZ	85001	1998-05-10	Male	2025-01-11 21:50:33	192.168.1.5	5	6000
6	Fiona	pass9012	fiona@example.com	6789012345	Philadelphia	PA	19019	1988-06-18	Female	2025-01-11 21:50:33	192.168.1.6	6	8000
7	George	pass3456	george@example.com	7890123456	San Antonio	TX	78201	1991-07-23	Male	2025-01-11 21:50:33	192.168.1.7	7	11000
8	Hannah	pass7890	hannah@example.com	8901234567	San Diego	CA	92101	1993-08-30	Female	2025-01-11 21:50:33	192.168.1.8	8	14000
9	Ivy	mypassword1	ivy@example.com	9012345678	Dallas	TX	75201	1994-09-05	Female	2025-01-11 21:50:33	192.168.1.9	9	18000
10	Jack	mypassword2	jack@example.com	0123456789	Austin	TX	73301	1996-10-15	Male	2025-01-11 21:50:33	192.168.1.10	10	22000
11	Karen	mypassword3	karen@example.com	1234509876	San Francisco	CA	94101	1997-11-20	Female	2025-01-11 21:50:33	192.168.1.11	11	27000

与 player_status 表通过 id 关联。

与 player_missions 表通过 id 关联。

与 raking_list 表通过 id 关联。

与 transactions 表通过 id 关联（作为买家或卖家）

2．活动表（activities ）

【设计理由】存储游戏中的活动信息，包括活动名称、类型、开始时间、结束时间、是否限时、奖励描述、完成人数和总参与人数。

字段名	数据类型	说明
id	INT	活动唯一标识（主键）
act_name	VARCHAR(50)	活动名称
act_type	VARCHAR(50)	活动类型
start_time	DATETIME	活动开始时间
end_time	DATETIME	活动结束时间
is_limited_time	BOOLEAN	是否限时活动
reward_description	TEXT	奖励描述
completed_users	INT	已完成活动的用户数量
total_participants	INT	活动总参与人数
created_at	TIMESTAMP	记录创建时间

3.角色表（chracters ）

【设计理由】存储游戏中的角色信息，包括角色名称、兵种类型、所属兵种、职业装备、特殊装备和特殊技能

字段名	数据类型	说明
id	INT	角色唯一标识（主键）
character_name	VARCHAR(100)	角色名称
class_type	VARCHAR(50)	兵种类型
subordinate_to	VARCHAR(100)	所属兵种

字段名	数据类型	说明
profession_equipment	TEXT	职业装备
special_equipment	TEXT	特殊装备
special_ability	TEXT	特殊技能

关联关系

- **与 classes 表通过 class_type 关联。**

4.兵种表（classes）

【设计理由】存储游戏中的兵种信息，包括兵种类型和描述。

字段名	数据类型	说明
class_type	VARCHAR(50)	兵种类型（主键）
description	TEXT	兵种描述

关联关系

- 与 chracters 表通过 class_type 关联。

5.武器表（weapons）

【设计理由】存储游戏中的武器信息，包括武器名称、类型、基础伤害、优势射程、伤害衰减百分比、射速、后坐力指数和散布指数。

字段名	数据类型	说明
weapon_id	INT	武器唯一标识（主键）
weapon_name	VARCHAR(100)	武器名称

字段名	数据类型	说明
weapon_type	VARCHAR(50)	武器种类
base_damage	INT	基础伤害
advantage_range	INT	优势射程（米）
damage_decay_percentage	DECIMAL(5, 2)	衰减百分比
rate_of_fire	INT	射速（每分钟）
recoil_index	DECIMAL(5, 2)	后坐力指数
spread_5m	DECIMAL(5, 2)	5 米散布指数
spread_15m	DECIMAL(5, 2)	15 米散布指数
spread_30m	DECIMAL(5, 2)	30 米散布指数

关联关系

- 与 player_status 表通过 weapon_id 关联。
- 与 player_weapon 表通过 weapon_id 关联。

6.配件类型表（attachments_type）

【设计理由】存储配件的类型信息，包括类型 ID 和类型名称。

字段数据类型

字段名	数据类型	说明
type_id	INT	类型 ID（主键）
type_name	VARCHAR(50)	配件类型名称

关联关系

- 与 attachments 表通过 type_id 关联。

7.配件表（attachments ）

【设计理由】存储配件的详细信息，包括配件名称、后坐力控制、开镜速度、腰射准度、人机工效和准镜倍率。

字段名	数据类型	说明
attachment_id	INT	配件 ID（主键）
type_id	INT	配件类型 ID
name	VARCHAR(100)	配件名称
recoil_control	DECIMAL(5, 2)	后坐力控制（%）
horizontal_recoil_control	DECIMAL(5, 2)	水平后坐力控制（%）
vertical_recoil_control	DECIMAL(5, 2)	垂直后坐力控制（%）
ads_speed	DECIMAL(5, 2)	开镜速度（%）
hipfire_accuracy	DECIMAL(5, 2)	腰射准度（%）
ergonomics	DECIMAL(5, 2)	人机工效（%）
scope_magnification	DECIMAL(5, 2)	准镜倍率

关联关系

- 与 attachment_types 表通过 type_id 关联。
- 与 player_weapon 表通过 attachment_id 关联。

8.装备类型表（equipment_types ）

【设计理由】存储装备的类型信息，包括类型 ID 和类型名称。

字段数据类型

字段名	数据类型	说明
type_id	INT	类型 ID（主键）
type_name	VARCHAR(50)	装备类型名称

关联关系

- 与 equipment_items 表通过 type_id 关联。

9．装备项目表(equipment_items)

【设计理由】存储装备的详细信息，包括装备名称、伤害、穿甲衰减、治疗量、治疗时间、持续时间、防护上限和防护衰减。

字段名	数据类型	说明
item_id	INT	装备 ID（主键）
type_id	INT	装备类型 ID
name	VARCHAR(100)	装备名称
damage	DECIMAL(5, 2)	伤害（伤害装备用）
armor_penetration	DECIMAL(5, 2)	穿甲衰减（伤害装备用）
healing_amount	DECIMAL(5, 2)	治疗量（治疗装备用）
healing_time	DECIMAL(5, 2)	治疗时间（治疗装备用）
duration	DECIMAL(5, 2)	持续时间（治疗装备用）
protection_limit	DECIMAL(5, 2)	防护上限（防护装备用）

字段名	数据类型	说明
protection_degradation	DECIMAL(5, 2)	防护衰减（防护装备用）

关联关系

- 与 equipment_types 表通过 type_id 关联。
- 与 player_status 表通过 item_id 关联。

10.任务表（missions ）

【设计理由】存储任务信息，包括任务名称、描述、奖励描述、开始时间和结束时间。

字段名	数据类型	说明
id	INT	任务唯一标识（主键）
mission_name	VARCHAR(50)	任务名称
description	TEXT	任务描述
reward_description	TEXT	奖励描述
start_time	DATETIME	任务开始时间
end_time	DATETIME	任务结束时间

关联关系

- 与 player_missions 表通过 id 关联。

11.玩家任务表（player_missions ）

【设计理由】存储玩家任务信息，包括玩家 ID、任务 ID、开始时间、

结束时间、进度和完成状态

字段名	数据类型	说明
id	INT	记录唯一标识（主键）
player_id	INT	玩家 ID
mission_id	INT	任务 ID
start_time	DATETIME	任务开始时间
end_time	DATETIME	任务结束时间
progress	INT	任务进度
completed	BOOLEAN	是否完成

关联关系

- 与 users 表通过 player_id 关联。
- 与 missions 表通过 mission_id 关联。

12.排行榜表（raking_list ）

【设计理由】存储排行榜信息，包括玩家 ID、用户名、分数、等级和经验值。

字段名	数据类型	说明
id	INT	记录唯一标识（主键）
player_id	INT	玩家 ID
username	VARCHAR(50)	用户名
score	INT	分数

字段名	数据类型	说明
level	INT	等级
experience	INT	经验值

关联关系

- **与 users 表通过 player_id 关联。**

13.交易信息表（transactions）

【设计理由】存储交易信息，包括交易 ID、买家 ID、卖家 ID、交易时间、总价和交易状态。

字段名	数据类型	说明
transaction_id	INT	交易唯一标识（主键）
buyer_id	INT	买家 ID
seller_id	INT	卖家 ID
transaction_time	TIMESTAMP	交易时间
total_price	DECIMAL(10, 2)	总价
status	ENUM	交易状态（pending, completed, cancelled）

关联关系

- 与 users 表通过 buyer_id 和 seller_id 关联。

14.玩家状态表（player_status ）

【设计理由】存储玩家状态信息，包括玩家 ID、生命值、负重能力、主武器 ID、副武器 ID、头部防护 ID 和身体防护 ID。

字段名	数据类型	说明
player_id	INT	玩家 ID（主键）
health	INT	生命值
weight_bearing_capacity	INT	负重能力
main_weapon_id	INT	主武器 ID
sub_weapon_id	INT	副武器 ID
head_protection_id	INT	头部防护 ID
body_protection_id	INT	身体防护 ID

关联关系

- 与 users 表通过 player_id 关联。
- 与 weapons 表通过 main_weapon_id 和 sub_weapon_id 关联。
- 与 equipment_items 表通过 head_protection_id 和 body_protection_id 关联。

15.玩家武器表（player_weapon ）

【设计理由】存储玩家武器信息，包括武器 ID、武器名称、武器类型、配件 ID 和配件类型 ID。

字段名	数据类型	说明
id	INT	记录唯一标识（主键）
weapon_id	INT	武器 ID
weapon_name	VARCHAR(100)	武器名称
weapon_type	VARCHAR(50)	武器类型
attechment_id	INT	配件 ID
type_id	INT	配件类型 ID

关联关系

- 与 weapons 表通过 weapon_id 关联。
- 与 attachments 表通过 attechment_id 关联。
- 与 attachment_types 表通过 type_id 关联。

16.战斗记录表（combat_records ）

【设计理由】存储战斗记录信息，包括战斗 ID、战斗时间、战斗地点、击中部位、胜利者 ID、失败者 ID、胜利者和失败者的武器及防护装备。

字段名	数据类型	说明
combat_id	INT	战斗记录 ID（主键）
combat_time	TIMESTAMP	战斗时间
location_latitude	DECIMAL(9, 6)	战斗地点纬度
location_longitude	DECIMAL(9, 6)	战斗地点经度

字段名	数据类型	说明
hit_part	VARCHAR(50)	击中部位
winner_id	INT	胜利者 ID
loser_id	INT	失败者 ID
winner_main_weapon_id	INT	胜利者主武器 ID
winner_sub_weapon_id	INT	胜利者副武器 ID
loser_main_weapon_id	INT	失败者主武器 ID
loser_sub_weapon_id	INT	失败者副武器 ID
winner_head_protection_id	INT	胜利者头部防护 ID
winner_body_protection_id	INT	胜利者身体防护 ID
loser_head_protection_id	INT	失败者头部防护 ID
loser_body_protection_id	INT	失败者身体防护 ID

关联关系

- 与 player_status 表通过 winner_id 和 loser_id 关联。
- 与 weapons 表通过 winner_main_weapon_id、winner_sub_weapon_id、loser_main_weapon_id 和 loser_sub_weapon_id 关联。
- 与 equipment_items 表通过 winner_head_protection_id、winner_body_protection_id、loser_head_protection_id 和 loser_body_protection_id 关联。

二、 数据操作

2.1 创建表 源代码

```
CREATE TABLE IF NOT EXISTS users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL,
    password VARCHAR(50) NOT NULL,
    email VARCHAR(50) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    city VARCHAR(50) NOT NULL,
    state VARCHAR(50) NOT NULL,
    postal_code VARCHAR(20) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender ENUM('Male', 'Female', 'Other') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ip_address VARCHAR(50) NOT NULL,
    level INT DEFAULT 1,
    experience INT DEFAULT 0
);
CREATE TABLE IF NOT EXISTS activities (
    id INT PRIMARY KEY AUTO_INCREMENT,
    act_name VARCHAR(50) NOT NULL,
    act_type VARCHAR(50) NOT NULL,
    start_time DATETIME NOT NULL,
    end_time DATETIME NOT NULL,
    is_limited_time BOOLEAN DEFAULT FALSE,
    reward_description TEXT,
    completed_users INT DEFAULT 0,          -- 已完成活动的用户数量
    total_participants INT DEFAULT 0,       -- 活动总参与人数
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- 记录创建时间
);

CREATE TABLE IF NOT EXISTS chracters (
    id INT PRIMARY KEY AUTO_INCREMENT,
    character_name VARCHAR(100) NOT NULL,          -- 角色名称
    class_type VARCHAR(50) NOT NULL,               -- 兵种类型
    subordinate_to VARCHAR(100),                   -- 该角色所属兵种
    profession_equipment TEXT,                     -- 职业装备（JSON 格式或逗号分隔）
    special_equipment TEXT,                        -- 特殊装备（JSON 格式或逗号分隔）
    special_ability TEXT,                          -- 特殊技能
    FOREIGN KEY (subordinate_to) REFERENCES classes(class_type)  -- 外键关联兵种表
```

```
);


-- 兵种表
CREATE TABLE IF NOT EXISTS classes (
    class_type VARCHAR(50) PRIMARY KEY,     -- 兵种类型
    description TEXT                        -- 兵种描述
);

-- 武器表

CREATE TABLE weapons (
    weapon_id INT PRIMARY KEY AUTO_INCREMENT,  -- 武器唯一标识
    weapon_name VARCHAR(100) NOT NULL,          -- 武器名称
    weapon_type VARCHAR(50) NOT NULL,           -- 武器种类（如“步枪”，“冲锋枪”等）
    base_damage INT NOT NULL,                   -- 基础伤害
    advantage_range INT NOT NULL,               -- 优势射程（米）
    damage_decay_percentage DECIMAL(5, 2) NOT NULL, -- 衰减百分比
    rate_of_fire INT NOT NULL,                  -- 射速（每分钟）
    recoil_index DECIMAL(5, 2) NOT NULL,        -- 后坐力指数
    spread_5m DECIMAL(5, 2) NOT NULL,           -- 5 米散布指数
    spread_15m DECIMAL(5, 2) NOT NULL,          -- 15 米散布指数
    spread_30m DECIMAL(5, 2) NOT NULL           -- 30 米散布指数
);

CREATE TABLE attachment_types (
    type_id INT PRIMARY KEY AUTO_INCREMENT, -- 类型 ID
    type_name VARCHAR(50) NOT NULL          -- 配件类型名称
);

CREATE TABLE attachments (
    attachment_id INT PRIMARY KEY AUTO_INCREMENT,  -- 配件 ID
    type_id INT NOT NULL,                           -- 配件类型（外键关联 attachment_types）
    name VARCHAR(100) NOT NULL,                     -- 配件名称
    recoil_control DECIMAL(5, 2),                   -- 后坐力控制（%）
    horizontal_recoil_control DECIMAL(5, 2),        -- 水平后坐力控制（%）
    vertical_recoil_control DECIMAL(5, 2),          -- 垂直后坐力控制（%）
    ads_speed DECIMAL(5, 2),                        -- 开镜速度（%）
    hipfire_accuracy DECIMAL(5, 2),                 -- 腰射准度（%）
    ergonomics DECIMAL(5, 2),                       -- 人机工效（%）
    scope_magnification DECIMAL(5, 2),              -- 准镜倍率，仅准镜有效
    FOREIGN KEY (type_id) REFERENCES attachment_types(type_id)
);
CREATE TABLE equipment_types (
```

```
    type_id INT PRIMARY KEY AUTO_INCREMENT,  -- 类型 ID
    type_name VARCHAR(50) NOT NULL           -- 装备类型名称（伤害装备、治疗装备、防护装备）
);

CREATE TABLE equipment_items (
    item_id INT PRIMARY KEY AUTO_INCREMENT,          -- 装备 ID
    type_id INT NOT NULL,                            -- 装备类型（外键关联 equipment_types）
    name VARCHAR(100) NOT NULL,                      -- 装备名称
    damage DECIMAL(5, 2),                            -- 伤害（伤害装备用）
    armor_penetration DECIMAL(5, 2),                 -- 穿甲衰减（伤害装备用）
    healing_amount DECIMAL(5, 2),                    -- 治疗量（治疗装备用）
    healing_time DECIMAL(5, 2),                      -- 治疗时间（治疗装备用）
    duration DECIMAL(5, 2),                          -- 持续时间（治疗装备用）
    protection_limit DECIMAL(5, 2),                  -- 防护上限（防护装备用）
    protection_degradation DECIMAL(5, 2),            -- 防护衰减（防护装备用）
    FOREIGN KEY (type_id) REFERENCES equipment_types(type_id)
);
CREATE TABLE missions (
    id INT PRIMARY KEY AUTO_INCREMENT,
    mission_name VARCHAR(50) NOT NULL,
    description TEXT,
    reward_description TEXT,
    start_time DATETIME NOT NULL,
    end_time DATETIME NOT NULL
);

CREATE TABLE player_missions (
    id INT PRIMARY KEY AUTO_INCREMENT,
    player_id INT NOT NULL,
    mission_id INT NOT NULL,
    start_time DATETIME NOT NULL,
    end_time DATETIME NOT NULL,
    progress INT DEFAULT 0,
    completed BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (player_id) REFERENCES users(id),
    FOREIGN KEY (mission_id) REFERENCES missions(id)
);
CREATE TABLE raking_list (
    id INT PRIMARY KEY AUTO_INCREMENT,
    player_id INT NOT NULL,
    username VARCHAR(50) NOT NULL,
    score INT NOT NULL,
```

```
    level INT NOT NULL,
    experience INT NOT NULL,
    FOREIGN KEY (level) REFERENCES users(level),
    FOREIGN KEY (experience) REFERENCES users(experience),
    FOREIGN KEY (player_id) REFERENCES users(id)
);
CREATE TABLE transactions (
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,    -- 交易唯一标识
    buyer_id INT NOT NULL,                            -- 买家 ID
    seller_id INT NOT NULL,                           -- 卖家 ID
    transaction_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  -- 交易时间
    total_price DECIMAL(10, 2) NOT NULL,              -- 总价
    status ENUM('pending', 'completed', 'cancelled') NOT NULL, -- 交易状态（待处理、完成、取消）
    FOREIGN KEY (buyer_id) REFERENCES users(id),      -- 买家（关联用户表）
    FOREIGN KEY (seller_id) REFERENCES users(id)      -- 卖家（关联用户表）
);
CREATE TABLE player_status (
    player_id INT PRIMARY KEY,
    health INT DEFAULT 120,
    weight_bearing_capacity INT DEFAULT 120,
    main_weapon_id INT,
    sub_weapon_id INT,
    head_protection_id INT,
    body_protection_id INT,
    FOREIGN KEY (main_weapon_id) REFERENCES weapons(weapon_id),
    FOREIGN KEY (sub_weapon_id) REFERENCES weapons(weapon_id),
    FOREIGN KEY (head_protection_id) REFERENCES equipment_items(item_id),
    FOREIGN KEY (body_protection_id) REFERENCES equipment_items(item_id)
);

CREATE TABLE player_weapon(
    id INT PRIMARY KEY AUTO_INCREMENT,
    weapon_id INT NOT NULL,
    weapon_name VARCHAR(100) NOT NULL,
    weapon_type VARCHAR(50) NOT NULL,
    attechment_id INT,
    type_id INT NOT NULL,
    FOREIGN KEY (type_id) REFERENCES attachment_types(type_id),
    FOREIGN KEY (attechment_id) REFERENCES attachments(attachment_id),
    FOREIGN KEY (weapon_id) REFERENCES weapons(weapon_id)
);
CREATE TABLE combat_records (
    combat_id INT PRIMARY KEY AUTO_INCREMENT,         -- 战斗记录 ID
```

```
    combat_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- 战斗时间
    location_latitude DECIMAL(9, 6) NOT NULL,        -- 战斗地点经度
    location_longitude DECIMAL(9, 6) NOT NULL,       -- 战斗地点纬度
    hit_part VARCHAR(50),                            -- 击中部位（例如：头部、胸部等）
    winner_id INT NOT NULL,                          -- 击败的玩家 ID（关联玩家状态）
    loser_id INT NOT NULL,                           -- 被击败的玩家 ID（关联玩家状态）
    winner_main_weapon_id INT,                       -- 击败玩家的主武器 ID（关联玩家武
器）
    winner_sub_weapon_id INT,                        -- 击败玩家的副武器 ID（关联玩家武
器）
    loser_main_weapon_id INT,                        -- 被击败玩家的主武器 ID（关联玩家武
器）
    loser_sub_weapon_id INT,                         -- 被击败玩家的副武器 ID（关联玩家武
器）
    winner_head_protection_id INT,                   -- 击败玩家的头部防护装备 ID（关联装
备表）
    winner_body_protection_id INT,                   -- 击败玩家的身体防护装备 ID（关联装
备表）
    loser_head_protection_id INT,                    -- 被击败玩家的头部防护装备 ID（关联
装备表）
    loser_body_protection_id INT,                    -- 被击败玩家的身体防护装备 ID（关联
装备表）
    FOREIGN KEY (winner_id) REFERENCES player_status(player_id),    -- 外键：击败玩家
    FOREIGN KEY (loser_id) REFERENCES player_status(player_id),     -- 外键：被击败玩家
    FOREIGN KEY (winner_main_weapon_id) REFERENCES weapons(weapon_id), -- 外键：击
败玩家主武器
    FOREIGN KEY (winner_sub_weapon_id) REFERENCES weapons(weapon_id),  -- 外键：击
败玩家副武器
    FOREIGN KEY (loser_main_weapon_id) REFERENCES weapons(weapon_id),  -- 外键：被
击败玩家主武器
    FOREIGN KEY (loser_sub_weapon_id) REFERENCES weapons(weapon_id),   -- 外键：被
击败玩家副武器
    FOREIGN KEY (winner_head_protection_id) REFERENCES equipment_items(item_id), -- 外
键：击败玩家头部防护装备
    FOREIGN KEY (winner_body_protection_id) REFERENCES equipment_items(item_id), --
外键：击败玩家身体防护装备
    FOREIGN KEY (loser_head_protection_id) REFERENCES equipment_items(item_id),  -- 外
键：被击败玩家头部防护装备
    FOREIGN KEY (loser_body_protection_id) REFERENCES equipment_items(item_id)  -- 外
键：被击败玩家身体防护装备
);
```

2.2 查询语句

-- 1. 查询每个玩家的用户名、等级、经验值以及他们装备的主武器名称

```
SELECT u.username, u.level, u.experience, w.weapon_name AS main_weapon
FROM users u
JOIN player_status ps ON u.id = ps.player_id
JOIN weapons w ON ps.main_weapon_id = w.weapon_id;
```

Alice	1	500	AK47
Bob	2	1500	M4A1
Charlie	3	2500	FAMAS F1
Daisy	4	4000	Scar-L
Ethan	5	6000	Steyr AUG
Fiona	6	8000	F2000
George	7	11000	SIG SG 550
Hannah	8	14000	G3A3
Ivy	9	18000	M14
Jack	10	22000	FN SCAR-H

-- 2. 查询每个活动的名称、参与人数以及完成人数

```
SELECT act_name, total_participants, completed_users
FROM activities;
```

Operation Dawn	500	150
Supply Raid	700	300
Rec	120	50
Urban Warfare	800	200
Dark Zone Expedition	1200	400
Extraction Point Alpha	600	220
Midnight Siege	300	90
Scavenger Hunt	800	350
Frontline Defense	1000	500
High-Risk Loot Run	250	100
Final Stand	400	180

-- 3. 查询每个玩家的任务完成情况（任务名称、进度、是否完成）

```
SELECT u.username, m.mission_name, pm.progress, pm.completed
FROM player_missions pm
```

```
JOIN users u ON pm.player_id = u.id
JOIN missions m ON pm.mission_id = m.id;
```

username varchar	mission_name varchar	progress int	completed tinyint
Alice	Operation Blackout	100	1
Bob	Silent Strike	90	0
Charlie	High Value Target	100	1
Daisy	Supply Line Cutoff	80	0
Ethan	Hostage Rescue	100	1
Fiona	Defensive Perimeter	60	0
George	Recon in Force	100	1
Hannah	Sabotage Ops	70	0
Ivy	Overwatch	100	1
Jack	Deep Strike	80	1
Karen	Final Stand	100	1

```
-- 4. 查询每个玩家的总分数、等级和经验值，并按分数降序排列
SELECT username, score, level, experience
FROM raking_list
ORDER BY score DESC;
```

* username varchar(50)	* score int	* level int	* experience int
Karen	27000	11	27000
Jack	22000	10	22000
Ivy	18000	9	18000
Hannah	14000	8	14000
George	11000	7	11000
Fiona	8000	6	8000
Ethan	6000	5	6000
Daisy	4000	4	4000
Charlie	2500	3	2500
Bob	1500	2	1500
Alice	500	1	500

Columns: All (4)
Enter column name
username score level
experience

```
-- 5. 查询每个玩家的战斗记录数量
SELECT u.username, COUNT(cr.combat_id) AS combat_count
FROM combat_records cr
JOIN users u ON cr.winner_id = u.id
GROUP BY u.username;
```

username varchar	combat_count bigint
Alice	1
Bob	1
Charlie	1
Daisy	1
Ethan	1
Fiona	1
George	1
Hannah	1
Ivy	1
Jack	1

```
-- 6. 查询每个玩家的平均战斗胜利次数（使用子查询）
```

```
SELECT u.username,
        (SELECT COUNT(*)
         FROM combat_records cr
         WHERE cr.winner_id = u.id) AS win_count
FROM users u;
```

username varchar	win_count bigint
Alice	1
Bob	1
Charlie	1
Daisy	1
Ethan	1
Fiona	1
George	1
Hannah	1
Ivy	1
Jack	1
Karen	0

```
-- 7. 查询每个玩家的最高分数和对应的等级
SELECT username, MAX(score) AS max_score, level
FROM raking_list
GROUP BY username, level;
```

* username varchar(50)	max_score	* level int
Alice	500	1
Bob	1500	2
Charlie	2500	3
Daisy	4000	4
Ethan	6000	5
Fiona	8000	6
George	11000	7
Hannah	14000	8
Ivy	18000	9
Jack	22000	10
Karen	27000	11

```
-- 8. 查询每个玩家的装备情况（主武器、副武器、头部防护、身体防护）
SELECT u.username,
        w1.weapon_name AS main_weapon,
        w2.weapon_name AS sub_weapon,
        eh.name AS head_protection,
        eb.name AS body_protection
FROM player_status ps
JOIN users u ON ps.player_id = u.id
JOIN weapons w1 ON ps.main_weapon_id = w1.weapon_id
JOIN weapons w2 ON ps.sub_weapon_id = w2.weapon_id
JOIN equipment_items eh ON ps.head_protection_id = eh.item_id
JOIN equipment_items eb ON ps.body_protection_id = eb.item_id;
```

username varchar	main_weapon varchar	sub_weapon varchar	head_protection varchar	body_protection varchar
Alice	AK47	M16	手雷	手雷
Bob	M4A1	AK74M	C4炸药	C4炸药
Charlie	FAMAS F1	G36	燃烧瓶	燃烧瓶
Daisy	Scar-L	Tavor X95	1级子弹	1级子弹
Ethan	Steyr AUG	QBZ-95	2级子弹	2级子弹
Fiona	F2000	L85A2	3级子弹	3级子弹
George	SIG SG 550	M1A1	4级子弹	4级子弹
Hannah	G3A3	M1 Garand	5级子弹	5级子弹
Ivy	M14	M21	6级子弹	6级子弹
Jack	FN SCAR-H	K14	小治疗针	小治疗针

```
-- 9. 查询每个玩家的任务完成率（已完成任务数 / 总任务数）
SELECT u.username,
        COUNT(pm.mission_id) AS total_missions,
        SUM(CASE WHEN pm.completed = TRUE THEN 1 ELSE 0 END) AS completed_missions,
            (SUM(CASE WHEN pm.completed = TRUE THEN 1 ELSE 0 END) /
COUNT(pm.mission_id)) * 100 AS completion_rate
FROM player_missions pm
JOIN users u ON pm.player_id = u.id
GROUP BY u.username;
```

username varchar	total_missions bigint	completed_missions decimal	completion_rate decimal
Alice	1	1	100.0000
Bob	1	0	0.0000
Charlie	1	1	100.0000
Daisy	1	0	0.0000
Ethan	1	1	100.0000
Fiona	1	0	0.0000
George	1	1	100.0000
Hannah	1	0	0.0000
Ivy	1	1	100.0000
Jack	1	1	100.0000
Karen	1	1	100.0000

```
-- 10. 查询每个玩家的战斗胜利率（胜利次数 / 总次数）
SELECT u.username,
        (SELECT COUNT(*)
          FROM combat_records cr
          WHERE cr.winner_id = u.id) AS win_count,
```

```
        (SELECT COUNT(*)
          FROM combat_records cr
          WHERE cr.loser_id = u.id) AS lose_count
FROM users u;
```

username varchar	win_count bigint	lose_count bigint
Alice	1	1
Bob	1	1
Charlie	1	1
Daisy	1	1
Ethan	1	1
Fiona	1	1
George	1	1
Hannah	1	1
Ivy	1	1
Jack	1	1
Karen	0	0

多表联接查询：查询 1、3、5、8 使用了多表联接（JOIN），将多个表的数据关联起来。
子查询：查询 6、10 使用了子查询，嵌套查询结果。
分组统计：查询 5、7、9 使用了 GROUP BY 和聚合函数（如 COUNT、MAX），对数据进行分组统计。

三、性能优化

I 索引

3.1 users

理由

- level 和 experience 是常用于查询和排序的字段。
- 创建索引可以加快按等级或经验值查询和排序的速度。

```
CREATE INDEX idx_username ON users(username);
```

- 索引前：
- **type: ALL**：表示全表扫描，性能较差。
- **rows: 11**：扫描了 11 行数据。

```
+----+-------------+-------+------------+------+---------------+------+---------+------+------
+----------+-------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows |
filtered | Extra       |
+----+-------------+-------+------------+------+---------------+------+---------+------+------
+----------+-------------+
|  1 | SIMPLE      | users | NULL       | ALL  | NULL          | NULL | NULL    | NULL |   11 |
10.00 | Using where |
+----+-------------+-------+------------+------+---------------+------+---------+------+------
+----------+-------------+
```

索引后

type: ref：表示使用了索引，性能显著提升。

rows: 1：只扫描了 1 行数据

```
+----+-------------+-------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key           | key_len | ref
| rows | filtered | Extra |
+----+-------------+-------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
|  1 | SIMPLE      | users | NULL       | ref  | idx_level     | idx_level     | 4       | const
|    1 |   100.00 | NULL  |
+----+-------------+-------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
```

3.2 ctivities 表的索引

索引：idx_act_name

创建理由：

- act_name 是常用于查询的字段。
- 创建索引可以加快按活动名称查询的速度。

```
+----+-------------+------------+------------+------+---------------+------+---------+------
+------+----------+-------------+
| id | select_type | table      | partitions | type | possible_keys | key  | key_len | ref  | ro
ws | filtered | Extra       |
+----+-------------+------------+------------+------+---------------+------+---------+------
+------+----------+-------------+
|  1 | SIMPLE      | activities | NULL       | ALL  | NULL          | NULL | NULL    | NULL |
11 |    10.00 | Using where |
+----+-------------+------------+------------+------+---------------+------+---------+------
+------+----------+-------------+
```

- **type: ALL**：表示全表扫描，性能较差。
- **rows: 11**：扫描了 11 行数据。

```
+----+-------------+------------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
| id | select_type | table      | partitions | type | possible_keys | key           | key_len |
ref   | rows | filtered | Extra |
+----+-------------+------------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
|  1 | SIMPLE      | activities | NULL       | ref  | idx_act_name  | idx_act_name  | 202     |
const |    1 |   100.00 | NULL  |
+----+-------------+------------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
```

- **type: ref**：表示使用了索引，性能显著提升。
- **rows: 1**：只扫描了 1 行数据。

3. weapons 表的索引

索引：idx_weapon_type

创建理由：

- weapon_type 是常用于查询的字段。
- 创建索引可以加快按武器类型查询的速度。

```
+----+-------------+---------+------------+------+---------------+------+---------+------+------
+----------+-------------+
| id | select_type | table   | partitions | type | possible_keys | key  | key_len | ref  | rows
| filtered | Extra       |
+----+-------------+---------+------------+------+---------------+------+---------+------+------
+----------+-------------+
|  1 | SIMPLE      | weapons | NULL       | ALL  | NULL          | NULL | NULL    | NULL |   20
|    10.00 | Using where |
+----+-------------+---------+------------+------+---------------+------+---------+------+------
+----------+-------------+
```

- **type: ALL**：表示全表扫描，性能较差。

- **rows: 20**：扫描了 20 行数据。

```
+----+-------------+---------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
| id | select_type | table   | partitions | type | possible_keys | key           | key_len | ref
| rows | filtered | Extra |
+----+-------------+---------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
|  1 | SIMPLE      | weapons | NULL       | ref  | idx_weapon_type | idx_weapon_type | 202     |
const |    5 |   100.00 | NULL  |
+----+-------------+---------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
```

- **type: ref**：表示使用了索引，性能显著提升。
- **rows: 5**：只扫描了 5 行数据。

4. attachments 表的索引

索引：idx_attachment_name

创建理由：

- name 是常用于查询的字段。
- 创建索引可以加快按配件名称查询的速度。

```
+----+-------------+-------------+------------+------+---------------+------+---------+------
+------+----------+-------------+
| id | select_type | table       | partitions | type | possible_keys | key  | key_len | ref  |
ows | filtered | Extra       |
+----+-------------+-------------+------------+------+---------------+------+---------+------
+------+----------+-------------+
|  1 | SIMPLE      | attachments | NULL       | ALL  | NULL          | NULL | NULL    | NULL |
13 |    10.00 | Using where |
```

- **type: ALL**：表示全表扫描，性能较差。
- **rows: 13**：扫描了 13 行数据。

```
+----+-------------+-------------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
| id | select_type | table       | partitions | type | possible_keys | key           | key_len |
ref   | rows | filtered | Extra |
+----+-------------+-------------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
|  1 | SIMPLE      | attachments | NULL       | ref  | idx_attachment_name | idx_attachment_name
| 402     | const |    1 |   100.00 | NULL  |
+----+-------------+-------------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
```

- **type: ref**：表示使用了索引，性能显著提升。

- **rows: 1**：只扫描了 1 行数据。

5. missions 表的索引

索引：idx_mission_name

创建理由：

- mission_name 是常用于查询的字段。
- 创建索引可以加快按任务名称查询的速度。
-

```
+----+-------------+----------+------------+------+---------------+------+---------+------
+------+----------+-------------+
| id | select_type | table    | partitions | type | possible_keys | key  | key_len | ref  | rows
| filtered | Extra       |
+----+-------------+----------+------------+------+---------------+------+---------+------
+------+----------+-------------+
|  1 | SIMPLE      | missions | NULL       | ALL  | NULL          | NULL | NULL    | NULL |   11
|    10.00 | Using where |
+----+-------------+----------+------------+------+---------------+------+---------+------
+------+----------+-------------+
```

  type: ALL：表示全表扫描，性能较差。
- **rows: 11**：扫描了 11 行数据。

```
+----+-------------+----------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
| id | select_type | table    | partitions | type | possible_keys | key           | key_len | re
f   | rows | filtered | Extra |
+----+-------------+----------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
|  1 | SIMPLE      | missions | NULL       | ref  | idx_mission_name | idx_mission_name | 202
| const |    1 |   100.00 | NULL  |
+----+-------------+----------+------------+------+---------------+---------------+---------
+-------+------+----------+-------+
```

- **type: ref**：表示使用了索引，性能显著提升。
- **rows: 1**：只扫描了 1 行数据。

源代码：

CREATE INDEX idx_act_name ON activities(act_name);

CREATE INDEX idx_character_name ON chracters(character_name);

```
CREATE INDEX idx_class_description ON classes(description(100));

CREATE INDEX idx_weapon_type ON weapons(weapon_type);
CREATE INDEX idx_attachment_name ON attachments(name);
CREATE INDEX idx_equipment_name ON equipment_items(name);
CREATE INDEX idx_mission_name ON missions(mission_name);
CREATE INDEX idx_player_mission_progress ON player_missions(progress);
CREATE INDEX idx_combat_time ON combat_records(combat_time);
```

II 直方图

创建直方图可以帮助优化器更好地估计不同等级玩家的分布情况，从而优化查询计划。

```
ANALYZE TABLE users UPDATE HISTOGRAM ON level WITH 10 BUCKETS;
EXPLAIN SELECT COUNT(*) FROM users WHERE level = 5;
```

-- level 列表示玩家的等级，可能分布不均匀（例如，低等级玩家较多，高等级玩家较少）。

--type: ALL：表示全表扫描，性能较差。

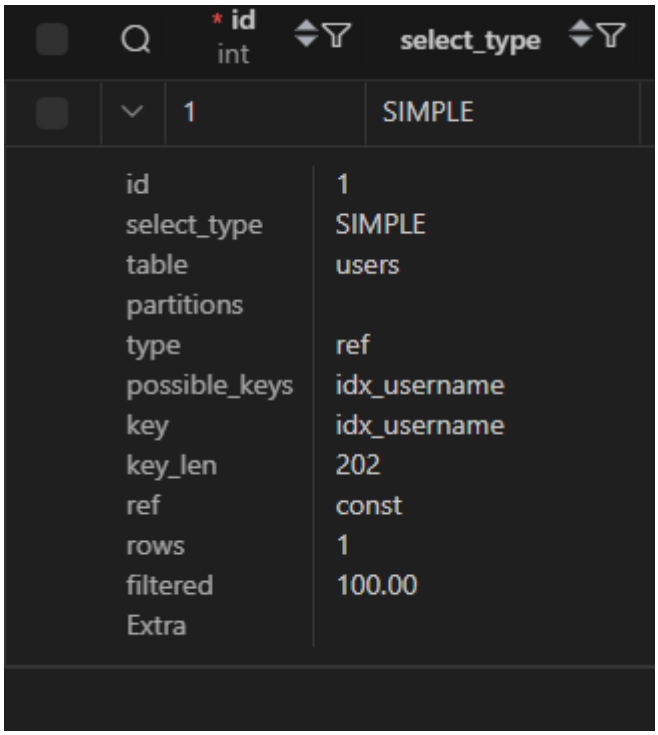
--rows: 11：扫描了 11 行数据。

--filtered: 10.00：优化器估计只有 10% 的数据满足条件

type: ref：表示使用了索引，性能显著提升。

rows: 1：只扫描了 1 行数据。

filtered: 100.00：优化器准确估计了满足条件的数据比例。



* id int	select_type
1	SIMPLE

id	1
select_type	SIMPLE
table	users
partitions	
type	ref
possible_keys	idx_username
key	idx_username
key_len	202
ref	const
rows	1
filtered	100.00
Extra	

	1	SIMPLE
id	1	
select_type	SIMPLE	
table	users	
partitions		
type	ref	
possible_keys	idx_level	
key	idx_level	
key_len	5	
ref	const	
rows	1	
filtered	100.00	
Extra	Using index	

四、数据库编程

4.1 五个储存函数

```
-- 1. 函数：计算玩家的战斗力，根据玩家的主武器基础伤害和副武器基础伤害，计算玩家的战斗力。
DELIMITER $$

CREATE FUNCTION CalculateCombatPower(player_id INT)
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE main_weapon_damage INT;
    DECLARE sub_weapon_damage INT;
    DECLARE combat_power INT;

    -- 获取主武器基础伤害
    SELECT base_damage INTO main_weapon_damage
    FROM weapons
    WHERE weapon_id = (SELECT main_weapon_id FROM player_status WHERE player_id = player_id);

    -- 获取副武器基础伤害
    SELECT base_damage INTO sub_weapon_damage
```

```
    FROM weapons
    WHERE weapon_id = (SELECT sub_weapon_id FROM player_status WHERE player_id = player_id);

    -- 计算战斗力
    SET combat_power = main_weapon_damage + sub_weapon_damage;

    RETURN combat_power;
END $$

DELIMITER ;

-- 查询玩家 1 的战斗力
SELECT CalculateCombatPower(1) AS combat_power;

-- 2. 函数：检查玩家是否完成所有任务
DELIMITER $$

CREATE FUNCTION CheckAllMissionsCompleted(player_id INT)
RETURNS BOOLEAN
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE total_missions INT;
    DECLARE completed_missions INT;

    -- 获取总任务数
    SELECT COUNT(*) INTO total_missions
    FROM player_missions
    WHERE player_id = player_id;

    -- 获取已完成任务数
    SELECT COUNT(*) INTO completed_missions
    FROM player_missions
    WHERE player_id = player_id AND completed = TRUE;

    -- 判断是否完成所有任务
    IF total_missions = completed_missions THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
END $$
```

```
DELIMITER ;

-- 检查玩家 1 是否完成所有任务
SELECT CheckAllMissionsCompleted(1) AS all_missions_completed;

-- 3. 函数：获取玩家的平均战斗胜利次数
DELIMITER $$

CREATE FUNCTION GetAverageWins(player_id INT)
RETURNS DECIMAL(5, 2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE total_wins INT;
    DECLARE total_combats INT;
    DECLARE average_wins DECIMAL(5, 2);

    -- 获取总胜利次数
    SELECT COUNT(*) INTO total_wins
    FROM combat_records
    WHERE winner_id = player_id;

    -- 获取总战斗次数
    SELECT COUNT(*) INTO total_combats
    FROM combat_records
    WHERE winner_id = player_id OR loser_id = player_id;

    -- 计算平均胜利次数
    IF total_combats > 0 THEN
        SET average_wins = total_wins / total_combats;
    ELSE
        SET average_wins = 0;
    END IF;

    RETURN average_wins;
END $$

DELIMITER ;

-- 查询玩家 1 的平均战斗胜利次数
SELECT GetAverageWins(1) AS average_wins;

-- 4. 函数：获取玩家的最高分数
DELIMITER $$
```

```
CREATE FUNCTION GetMaxScore(player_id INT)
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE max_score INT;

    -- 获取最高分数
    SELECT MAX(score) INTO max_score
    FROM raking_list
    WHERE player_id = player_id;

    RETURN max_score;
END $$

DELIMITER ;
-- 查询玩家 1 的最高分数
SELECT GetMaxScore(1) AS max_score;

-- 5. 函数：检查玩家是否装备了特定类型的武器
DELIMITER $$

CREATE FUNCTION CheckWeaponTypeEquipped(player_id INT, weapon_type VARCHAR(50))
RETURNS BOOLEAN
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE weapon_count INT;

    -- 检查玩家是否装备了特定类型的武器
    SELECT COUNT(*) INTO weapon_count
    FROM player_weapon pw
    JOIN weapons w ON pw.weapon_id = w.weapon_id
    WHERE pw.weapon_id IN (
        SELECT main_weapon_id FROM player_status WHERE player_id = player_id
        UNION
        SELECT sub_weapon_id FROM player_status WHERE player_id = player_id
    ) AND w.weapon_type = weapon_type;

    IF weapon_count > 0 THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
```

```
    END IF;
END $$

DELIMITER ;

-- 检查玩家 1 是否装备了步枪
SELECT CheckWeaponTypeEquipped(1, '步枪') AS is_rifle_equipped;
```

4.2 创建储存过程

```
-- 1. 存储过程: 更新玩家的经验值: 根据玩家完成任务的经验奖励，更新玩家的经验值。
DELIMITER $$

CREATE PROCEDURE UpdatePlayerExperience(IN player_id INT, IN experience_reward INT)
BEGIN
    -- 更新玩家的经验值
    UPDATE users
    SET experience = experience + experience_reward
    WHERE id = player_id;
END $$

DELIMITER ;
-- 更新玩家 1 的经验值，增加 500 点
CALL UpdatePlayerExperience(1, 500);
```

```
-- 查询玩家 1 的当前经验值
SELECT experience FROM users WHERE id = 1;

-- 2. 存储过程：重置玩家的任务进度：将某个玩家的所有任务进度重置为 0，并将完成状态设置为 FALSE
DELIMITER $$

CREATE PROCEDURE ResetPlayerMissions(IN player_id INT)
BEGIN
    -- 重置玩家的任务进度
    UPDATE player_missions
    SET progress = 0, completed = FALSE
    WHERE player_id = player_id;
END $$

DELIMITER ;
-- 重置玩家 1 的任务进度
CALL ResetPlayerMissions(1);

-- 查询玩家 1 的任务进度
SELECT * FROM player_missions WHERE player_id = 1;
```

-- 3. 存储过程：删除玩家的战斗记录：删除某个玩家的所有战斗记录

```
DELIMITER $$

CREATE PROCEDURE DeletePlayerCombatRecords(IN player_id INT)
BEGIN
    -- 删除玩家的战斗记录
    DELETE FROM combat_records
    WHERE winner_id = player_id OR loser_id = player_id;
END $$

DELIMITER ;

-- 删除玩家 1 的所有战斗记录
CALL DeletePlayerCombatRecords(1);

-- 查询玩家 1 的战斗记录
SELECT * FROM combat_records WHERE winner_id = 1 OR loser_id = 1;
```

-- 4. 存储过程：统计每个玩家的任务完成情况：统计每个玩家的总任务数、已完成任务数和任务完成率

```
DELIMITER $$

CREATE PROCEDURE GetPlayerMissionStats()
BEGIN
    -- 统计每个玩家的任务完成情况
    SELECT u.username,
           COUNT(pm.mission_id) AS total_missions,
           SUM(CASE WHEN pm.completed = TRUE THEN 1 ELSE 0
END) AS completed_missions,
           (SUM(CASE WHEN pm.completed = TRUE THEN 1 ELSE 0
END) / COUNT(pm.mission_id)) * 100 AS completion_rate
    FROM player_missions pm
    JOIN users u ON pm.player_id = u.id
    GROUP BY u.username;
END $$

DELIMITER ;

-- 查询所有玩家的任务完成情况
CALL GetPlayerMissionStats();

-- 5. 存储过程：为玩家分配新任务：为某个玩家分配一个新任务，
```

并设置任务的开始时间和结束时间。

```
DELIMITER $$

CREATE PROCEDURE AssignNewMission(IN player_id INT, IN mission_id INT, IN start_time DATETIME, IN end_time DATETIME)
BEGIN
    -- 为玩家分配新任务
    INSERT INTO player_missions (player_id, mission_id, start_time, end_time, progress, completed)
    VALUES (player_id, mission_id, start_time, end_time, 0, FALSE);
END $$

DELIMITER ;

-- 为玩家 1 分配任务 1，开始时间为当前时间，结束时间为 1 小时后
CALL AssignNewMission(1, 1, NOW(), DATE_ADD(NOW(), INTERVAL 1 HOUR));

-- 查询玩家 1 的任务分配情况
SELECT * FROM player_missions WHERE player_id = 1;
```

4.3 创建触发器

-- 1. users 表 在插入新用户之前，检查用户名是否已存在。如果用户名已存在，则阻止插入并抛出错误。

```
DELIMITER $$

CREATE TRIGGER before_insert_users
BEFORE INSERT ON users
FOR EACH ROW
BEGIN
    DECLARE user_count INT;

    -- 检查用户名是否已存在
    SELECT COUNT(*) INTO user_count
    FROM users
    WHERE username = NEW.username;

    -- 如果用户名已存在，抛出错误
    IF user_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Username already exists';
    END IF;
END $$
```

```
DELIMITER ;

-- 2. activities 表:在插入新活动之前，检查活动名称是否已存在。如果活动名称已存在，则阻止插入并抛出错误
DELIMITER $$

CREATE TRIGGER before_insert_activities
BEFORE INSERT ON activities
FOR EACH ROW
BEGIN
    DECLARE activity_count INT;

    -- 检查活动名称是否已存在
    SELECT COUNT(*) INTO activity_count
    FROM activities
    WHERE act_name = NEW.act_name;

    -- 如果活动名称已存在，抛出错误
    IF activity_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Activity name already exists';
```

```
    END IF;
END $$

DELIMITER ;

-- 3. chracters 表:在插入新角色之前，检查角色名称是否已存在。如果角色名称已存在，则阻止插入并抛出错误。
DELIMITER $$

CREATE TRIGGER before_insert_characters
BEFORE INSERT ON chracters
FOR EACH ROW
BEGIN
    DECLARE character_count INT;

    -- 检查角色名称是否已存在
    SELECT COUNT(*) INTO character_count
    FROM chracters
    WHERE character_name = NEW.character_name;

    -- 如果角色名称已存在，抛出错误
    IF character_count > 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Character name already exists';
    END IF;
END $$

DELIMITER ;

-- 4. classes 表:在插入新兵种之前，检查兵种类型是否已存在。如果兵种类型已存在，则阻止插入并抛出错误
DELIMITER $$

CREATE TRIGGER before_insert_classes
BEFORE INSERT ON classes
FOR EACH ROW
BEGIN
    DECLARE class_count INT;

    -- 检查兵种类型是否已存在
    SELECT COUNT(*) INTO class_count
    FROM classes
    WHERE class_type = NEW.class_type;
```

```
    -- 如果兵种类型已存在，抛出错误
    IF class_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Class type already exists';
    END IF;
END $$

DELIMITER ;

-- 5. missions 表:在插入新任务之前，检查任务名称是否已存在。如果任务名称已存在，则阻止插入并抛出错误。
DELIMITER $$

CREATE TRIGGER before_insert_missions
BEFORE INSERT ON missions
FOR EACH ROW
BEGIN
    DECLARE mission_count INT;

    -- 检查任务名称是否已存在
    SELECT COUNT(*) INTO mission_count
    FROM missions
```

```
    WHERE mission_name = NEW.mission_name;

    -- 如果任务名称已存在，抛出错误
    IF mission_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Mission name already exists';
    END IF;
END $$
```

4.4 创建视图

```
-- 1 player_weapon_details 显示每个玩家的武器详细信息，包括主武器、副武器及其配件。
CREATE VIEW player_weapon_details AS
SELECT
    u.username,
    w1.weapon_name AS main_weapon,
    w2.weapon_name AS sub_weapon,
    a1.name AS main_weapon_attachment,
    a2.name AS sub_weapon_attachment
FROM player_status ps
JOIN users u ON ps.player_id = u.id
JOIN weapons w1 ON ps.main_weapon_id = w1.weapon_id
```

```
JOIN weapons w2 ON ps.sub_weapon_id = w2.weapon_id
LEFT JOIN attachments a1 ON ps.main_weapon_id = a1.attachment_id
LEFT JOIN attachments a2 ON ps.sub_weapon_id = a2.attachment_id;

SELECT * FROM player_weapon_details;

-- 2 player_mission_progress 显示每个玩家的任务进度，包括任务名称、进度百分比和完成状态。
CREATE VIEW player_mission_progress AS
SELECT
    u.username,
    m.mission_name,
    pm.progress,
    pm.completed
FROM player_missions pm
JOIN users u ON pm.player_id = u.id
JOIN missions m ON pm.mission_id = m.id;
SELECT * FROM player_mission_progress;

-- 3 combat_summary 显示每个玩家的战斗总结，包括胜利次数、失败次数和胜率。
CREATE VIEW combat_summary AS
```

```
SELECT
    u.username,
     COUNT(CASE WHEN cr.winner_id = u.id THEN 1 END) AS win_count,
    COUNT(CASE WHEN cr.loser_id = u.id THEN 1 END) AS lose_count,
    ROUND(COUNT(CASE WHEN cr.winner_id = u.id THEN 1 END) / COUNT(*) * 100, 2) AS win_rate
FROM combat_records cr
JOIN users u ON cr.winner_id = u.id OR cr.loser_id = u.id
GROUP BY u.username;
SELECT * FROM combat_summary;

-- 4 activity_participation 显示每个活动的参与情况，包括活动名称、总参与人数和完成人数。
CREATE VIEW activity_participation AS
SELECT
    act_name,
    total_participants,
    completed_users
FROM activities;
SELECT * FROM activity_participation;
```

--- 5 player_equipment_summary 显示每个玩家的装备总结，包括主武器、副武器、头部防护和身体防护。

```
CREATE VIEW player_equipment_summary AS
SELECT
    u.username,
    w1.weapon_name AS main_weapon,
    w2.weapon_name AS sub_weapon,
    eh.name AS head_protection,
    eb.name AS body_protection
FROM player_status ps
JOIN users u ON ps.player_id = u.id
JOIN weapons w1 ON ps.main_weapon_id = w1.weapon_id
JOIN weapons w2 ON ps.sub_weapon_id = w2.weapon_id
JOIN equipment_items eh ON ps.head_protection_id = eh.item_id
JOIN equipment_items eb ON ps.body_protection_id = eb.item_id;
SELECT * FROM player_equipment_summary;
```

祝老师新年快乐哇!!